

C++ Standard Library Issues Resolved Directly In Jacksonville

Doc. no. P1003R0

Date: Revised 2018-03-16 at 17:30:30 UTC

Project: Programming Language C++

Reply to: Marshall Clow <lwgchair@gmail.com>

Immediate Issues

[2946^{\(i\)}](#). LWG 2758's resolution missed further corrections

Section: 24.3.2.2 [[string.cons](#)], 24.3.2.6.2 [[string.append](#)], 24.3.2.6.3 [[string.assign](#)], 24.3.2.7 [[string.ops](#)] **Status:** [Immediate](#) **Submitter:** Daniel Krügler
Opened: 2017-03-17 **Last modified:** 2018-03-16

Priority: 2

View other [active issues](#) in [[string.cons](#)].

View all other [issues](#) in [[string.cons](#)].

View all issues with [Immediate](#) status.

Discussion:

LWG [2758](#) corrected newly introduced ambiguities of `std::string::assign` and other functions that got new overloads taking a `basic_string_view` as argument, but the assignment operator of `basic_string` and other functions taking a parameter of type `basic_string_view<charT, traits>` were not corrected. Similar to the previous issue the following operations lead now to an ambiguity as well:

```
#include <string>

int main()
{
    std::string s({"abc", 1});
    s = {"abc", 1};
    s += {"abc", 1};
    s.append({"abc", 1});
    s.assign({"abc", 1});
    s.insert(0, {"abc", 1});
    s.replace(0, 1, {"abc", 1});
    s.replace(s.cbegin(), s.cbegin(), {"abc", 1});
    s.find({"abc", 1});
    s.rfind({"abc", 1});
    s.find_first_of({"abc", 1});
    s.find_last_of({"abc", 1});
    s.find_first_not_of({"abc", 1});
    s.find_last_not_of({"abc", 1});
    s.compare({"abc", 1});
    s.compare(0, 1, {"abc", 1});
}
```

The right fix is to convert *all* member functions taken a `basic_string_view<charT, traits>` parameter into constrained function templates.

When doing so, it turns out that there occurs an additional problem: The functions that had been massaged by LWG [2758](#) are all functions that are not specified to be `noexcept`, but the wider range of "string operation" functions taking a `basic_string_view<charT, traits>` parameter are mostly `noexcept` because they had a wide contract. Now with the approach of LWG [2758](#), there are all types allowed that are *convertible* to `basic_string_view<charT, traits>`, but the conversion occurs now in the function body, not outside of it. So, if these conversion *would* be potentially exception-throwing, this would lead to a call to `std::terminate`, which is a semantic change compared to the previous specification. There are several options to handle this situation:

1. Ignore that and let `std::terminate` come into action. This is a different way of saying that we impose the requirement of a nothrowing operation.
2. Remove `noexcept` from all the affected functions.
3. Make these functions conditionally `noexcept`.

The submitter has a personal preference for option (3), except that this would complicate the wording a bit, because unfortunately there exists yet no trait `std::is_nothrow_convertible` (See LWG [2040](#)). A seemingly low-hanging fruit would be the attempt to use `std::is_nothrow_constructible` instead, but this trait describes a potentially different initialization context and is therefore inappropriate. Option (1) would conserve the existing `noexcept` guarantee for all non-throwing conversions, but now these functions become narrow-contract functions and at least according to the [current noexcept guidelines](#) such functions should *not* be marked as `noexcept`. But there are exceptions possible for that rule, and the initially suggested

proposed wording below argues that this exception is reasonable here, because the required wording fixes just an unintended side-effects of transforming the functions into functions templates, but it doesn't intend to change the actual functionality.

Some of the below suggested overload exclusion constraints technically don't require the additional `is_convertible_v<const T&, const charT*> == false` requirement, but the submitter of this issue suggests a more advanced approach that should be applied in a synchronous wording adjustment combined with the existing LWG [2758](#) wording: It would presumably life easier for implementations (which are allowed to provide additional member function overloads as conforming extensions), when we would define a mini requirement set for template parameter type `T` below:

- `is_convertible_v<const T&, basic_string_view<charT, traits>>` is true.
- `is_convertible_v<const T&, const charT*>` is false.
- The implicit conversion to `basic_string_view<charT, traits>` shall not throw an exception.

But the corresponding slightly revised wording taking advantage of this "concept-like" requirements set will not be suggested before the upcoming working draft has been published to allow a simpler coordinated adjustment together with the LWG [2758](#) wording.

It should also be noted that these changes have impact on deduction behaviour and therefore may require further adjustments of the deduction rules.

[2017-07-13, Toronto]

LWG preferred to remove in all functions with added nothrow constraints the `noexcept` specifier (and the constraint) and to possibly improve the situation later.

Previous resolution [SUPERSEDED]:

This wording is relative to [N4640](#).

1. Edit 24.3.2 [\[basic.string\]](#), class template `basic_string` synopsis, as indicated:

```
[...]
// 21.3.2.2, construct/copy/destroy
[...]
template<class T>
explicit basic_string(basic_string_view<charT, traits> svconst T& t,
                    const Allocator& a = Allocator());
[...]
template<class T>
basic_string& operator=(basic_string_view<charT, traits> svconst T& t);
basic_string& operator=(const charT* s);
[...]

// 21.3.2.6, modifiers
[...]
template<class T>
basic_string& operator+=(basic_string_view<charT, traits> svconst T& t);
[...]
template<class T>
basic_string& append(basic_string_view<charT, traits> svconst T& t);
[...]
template<class T>
basic_string& assign(basic_string_view<charT, traits> svconst T& t);
[...]
template<class T>
basic_string& insert(size_type pos, basic_string_view<charT, traits> svconst T& t);
[...]
template<class T>
basic_string& replace(size_type pos1, size_type n1,
                    basic_string_view<charT, traits> svconst T& t);
[...]
template<class T>
basic_string& replace(const_iterator i1, const_iterator i2,
                    basic_string_view<charT, traits> svconst T& t);
[...]

// 21.3.2.7, string operations
[...]
template<class T>
size_type find(basic_string_view<charT, traits> svconst T& t,
              size_type pos = 0) const noexcept;
[...]
template<class T>
size_type rfind(basic_string_view<charT, traits> svconst T& t,
               size_type pos = npos) const noexcept;
[...]
template<class T>
size_type find_first_of(basic_string_view<charT, traits> svconst T& t,
                       size_type pos = 0) const noexcept;
[...]
template<class T>
size_type find_last_of(basic_string_view<charT, traits> svconst T& t,
```

```

        size_type pos = npos) const noexcept;
[...]  

template<class T>  

size_type find_first_not_of(basic_string_view<charT, traits> svconst T& t,  

                           size_type pos = 0) const noexcept;  

[...]  

template<class T>  

size_type find_last_not_of (basic_string_view<charT, traits> svconst T& t,  

                           size_type pos = npos) const noexcept;  

[...]  

template<class T>  

int compare(basic_string_view<charT, traits> svconst T& t) const noexcept;  

[...]  

template<class T>  

int compare(size_type pos1, size_type n1, basic_string_view<charT, traits> svconst T& t) const;  

[...]
```

2. Edit 24.3.2.2 [\[string.cons\]](#) as indicated:

```

template<class T>  

explicit basic_string(basic_string_view<charT, traits> svconst T& t,  

                    const Allocator& a = Allocator());  
  

-9- Effects: Same as basic_string(sv.data(), sv.size(), a).Creates a variable, sv, as if by  

basic_string_view<charT, traits> sv = t; and then behaves the same as  

basic_string(sv.data(), sv.size(), a).  
  

-?- Remarks: This constructor shall not participate in overload resolution unless  

is convertible v<const T&, basic_string_view<charT, traits>> is true and  

is convertible v<const T&, const charT*> is false.  
  

[...]  
  

template<class T>  

basic_string& operator=(basic_string_view<charT, traits> svconst T& t);  
  

-25- Effects: Equivalent to:  
  

{  

    basic_string_view<charT, traits> sv = t;  

    return assign(sv);  

}  
  

-?- Remarks: This function shall not participate in overload resolution unless is convertible v<const  

T&, basic_string_view<charT, traits>> is true and is convertible v<const T&, const  

charT*> is false.
```

3. Edit 24.3.2.6.1 [\[string.op+=\]](#) as indicated:

```

template<class T>  

basic_string& operator+=(basic_string_view<charT, traits> svconst T& t);  
  

-3- Effects: Creates a variable, sv, as if by basic_string_view<charT, traits> sv = t; and then  

calls append(sv).  
  

-4- Returns: *this.  
  

-?- Remarks: This function shall not participate in overload resolution unless is convertible v<const  

T&, basic_string_view<charT, traits>> is true and is convertible v<const T&, const  

charT*> is false.
```

4. Edit 24.3.2.6.2 [\[string.append\]](#) as indicated:

```

template<class T>  

basic_string& append(basic_string_view<charT, traits> svconst T& t);  
  

-6- Effects: Equivalent to:  
  

{  

    basic_string_view<charT, traits> sv = t;  

    return append(sv.data(), sv.size());  

}  
  

-?- Remarks: This function shall not participate in overload resolution unless is convertible v<const  

T&, basic_string_view<charT, traits>> is true and is convertible v<const T&, const  

charT*> is false.
```

5. Edit 24.3.2.6.3 [\[string.assign\]](#) as indicated:

```

template<class T>  

basic_string& assign(basic_string_view<charT, traits> svconst T& t);
```

-8- *Effects*: Equivalent to:

```
{  
    basic_string_view<charT, traits> sv = t;  
    return assign(sv.data(), sv.size());  
}
```

-?- Remarks: This function shall not participate in overload resolution unless $\text{is_convertible } v<\text{const } T\&, \text{ basic_string_view}<\text{charT}, \text{ traits}>>$ is true and $\text{is_convertible } v<\text{const } T\&, \text{ const charT*}>$ is false.

6. Edit 24.3.2.6.4 [\[string.insert\]](#) as indicated:

```
template<class T>  
basic_string& insert(size_type pos, basic_string_view<charT, traits> sv const T& t);
```

-5- *Effects*: Equivalent to:

```
{  
    basic_string_view<charT, traits> sv = t;  
    return insert(pos, sv.data(), sv.size());  
}
```

-?- Remarks: This function shall not participate in overload resolution unless $\text{is_convertible } v<\text{const } T\&, \text{ basic_string_view}<\text{charT}, \text{ traits}>>$ is true and $\text{is_convertible } v<\text{const } T\&, \text{ const charT*}>$ is false.

7. Edit 24.3.2.6.6 [\[string.replace\]](#) as indicated:

```
template<class T>  
basic_string& replace(size_type pos1, size_type n1,  
    basic_string_view<charT, traits> sv const T& t);
```

-5- *Effects*: Equivalent to:

```
{  
    basic_string_view<charT, traits> sv = t;  
    return replace(pos1, n1, sv.data(), sv.size());  
}
```

-?- Remarks: This function shall not participate in overload resolution unless $\text{is_convertible } v<\text{const } T\&, \text{ basic_string_view}<\text{charT}, \text{ traits}>>$ is true and $\text{is_convertible } v<\text{const } T\&, \text{ const charT*}>$ is false.

[...]

```
template<class T>  
basic_string& replace(const_iterator i1, const_iterator i2,  
    basic_string_view<charT, traits> sv const T& t);
```

-21- *Requires*: $[\text{begin}(), i1)$ and $[i1, i2)$ are valid ranges.

-22- *Effects*: Creates a variable, sv , as if by $\text{basic_string_view}<\text{charT}, \text{ traits}> \text{ sv} = t$; and then
calls $\text{replace}(i1 - \text{begin}(), i2 - i1, sv)$.

-23- *Returns*: $*\text{this}$.

-?- Remarks: This function shall not participate in overload resolution unless $\text{is_convertible } v<\text{const } T\&, \text{ basic_string_view}<\text{charT}, \text{ traits}>>$ is true and $\text{is_convertible } v<\text{const } T\&, \text{ const charT*}>$ is false.

8. Edit 24.3.2.7.2 [\[string.find\]](#) as indicated:

```
template<class T>  
size_type find(basic_string_view<charT, traits> sv const T& t, size_type pos = 0) const noexcept;
```

-?- Requires: The initialization of sv , as specified below, shall not throw an exception.

-1- *Effects*: Creates a variable, sv , as if by $\text{basic_string_view}<\text{charT}, \text{ traits}> \text{ sv} = t$; and then
determines the lowest position $xpos$, if possible, such that both of the following conditions hold: [...]

-2- *Returns*: $xpos$ if the function can determine such a value for $xpos$. Otherwise, returns $npos$.

-?- Remarks: This function shall not participate in overload resolution unless $\text{is_convertible } v<\text{const } T\&, \text{ basic_string_view}<\text{charT}, \text{ traits}>>$ is true and $\text{is_convertible } v<\text{const } T\&, \text{ const charT*}>$ is false.

9. Edit 24.3.2.7.3 [\[string.rfind\]](#) as indicated:

```
template<class T>
size_type rfind(basic_string_view<charT, traits> svconst T& t, size_type pos = npos) const noexcept;
```

-?- Requires: The initialization of sv, as specified below, shall not throw an exception.

-1- Effects: Creates a variable, sv, as if by `basic_string_view<charT, traits> sv = t;` and then dDdetermines the highest position xpos, if possible, such that both of the following conditions hold: [...]

-2- Returns: xpos if the function can determine such a value for xpos. Otherwise, returns npos.

-?- Remarks: This function shall not participate in overload resolution unless `is_convertible_v<const T&, basic_string_view<charT, traits>>` is true and `is_convertible_v<const T&, const charT*>` is false.

10. Edit 24.3.2.7.4 [\[string.find.first.of\]](#) as indicated:

```
template<class T>
size_type find_first_of(basic_string_view<charT, traits> svconst T& t, size_type pos = 0) const noexcept;
```

-?- Requires: The initialization of sv, as specified below, shall not throw an exception.

-1- Effects: Creates a variable, sv, as if by `basic_string_view<charT, traits> sv = t;` and then dDdetermines the lowest position xpos, if possible, such that both of the following conditions hold: [...]

-2- Returns: xpos if the function can determine such a value for xpos. Otherwise, returns npos.

-?- Remarks: This function shall not participate in overload resolution unless `is_convertible_v<const T&, basic_string_view<charT, traits>>` is true and `is_convertible_v<const T&, const charT*>` is false.

11. Edit 24.3.2.7.5 [\[string.find.last.of\]](#) as indicated:

```
template<class T>
size_type find_last_of(basic_string_view<charT, traits> svconst T& t, size_type pos = npos) const noexcept;
```

-?- Requires: The initialization of sv, as specified below, shall not throw an exception.

-1- Effects: Creates a variable, sv, as if by `basic_string_view<charT, traits> sv = t;` and then dDdetermines the highest position xpos, if possible, such that both of the following conditions hold: [...]

-2- Returns: xpos if the function can determine such a value for xpos. Otherwise, returns npos.

-?- Remarks: This function shall not participate in overload resolution unless `is_convertible_v<const T&, basic_string_view<charT, traits>>` is true and `is_convertible_v<const T&, const charT*>` is false.

12. Edit 24.3.2.7.6 [\[string.find.first.not.of\]](#) as indicated:

```
template<class T>
size_type find_first_not_of(basic_string_view<charT, traits> svconst T& t,
                           size_type pos = 0) const noexcept;
```

-?- Requires: The initialization of sv, as specified below, shall not throw an exception.

-1- Effects: Creates a variable, sv, as if by `basic_string_view<charT, traits> sv = t;` and then dDdetermines the lowest position xpos, if possible, such that both of the following conditions hold: [...]

-2- Returns: xpos if the function can determine such a value for xpos. Otherwise, returns npos.

-?- Remarks: This function shall not participate in overload resolution unless `is_convertible_v<const T&, basic_string_view<charT, traits>>` is true and `is_convertible_v<const T&, const charT*>` is false.

13. Edit 24.3.2.7.7 [\[string.find.last.not.of\]](#) as indicated:

```
template<class T>
size_type find_last_not_of(basic_string_view<charT, traits> svconst T& t,
                           size_type pos = npos) const noexcept;
```

-?- Requires: The initialization of sv, as specified below, shall not throw an exception.

-1- Effects: Creates a variable, sv, as if by `basic_string_view<charT, traits> sv = t;` and then dDdetermines the highest position xpos, if possible, such that both of the following conditions hold: [...]

-2- Returns: xpos if the function can determine such a value for xpos. Otherwise, returns npos.

-?- Remarks: This function shall not participate in overload resolution unless `is_convertible_v<const T&, basic_string_view<charT, traits>>` is true and `is_convertible_v<const T&, const`

`charT*> is false.`

14. Edit 24.3.2.7.9 [\[string.compare\]](#) as indicated:

```
template<class T>
int compare(basic_string_view<charT, traits> svconst T& t) const noexcept;
```

`-?- Requires: The initialization of sv, as specified below, shall not throw an exception.`

-1- Effects: `Creates a variable, sv, as if by basic_string_view<charT, traits> sv = t; and then determines the effective length rlen of the strings to compare as the smaller of size() and sv.size(). The function then compares the two strings by calling traits::compare(data(), sv.data(), rlen).`

-2- Returns: The nonzero result if the result of the comparison is nonzero. Otherwise, returns a value as indicated in Table 63.

[...]

`-?- Remarks: This function shall not participate in overload resolution unless is_convertible_v<const T&, basic_string_view<charT, traits>> is true and is_convertible_v<const T&, const charT*> is false.`

```
template<class T>
int compare(size_type pos1, size_type n1, basic_string_view<charT, traits> svconst T& t) const;
```

-3- Effects: Equivalent to:

```
{
    basic_string_view<charT, traits> sv = t;
    return basic_string_view<charT, traits>(data(), size()).substr(pos1, n1).compare(sv);
}
```

`-?- Remarks: This function shall not participate in overload resolution unless is_convertible_v<const T&, basic_string_view<charT, traits>> is true and is_convertible_v<const T&, const charT*> is false.`

[2017-07-14, Toronto, Daniel refines wording]

To balance the loss of information about the removed noexcept specifications, all affected functions should get a new *Throws:* element saying that they won't throw unless this is caused by the conversion to the local `basic_string_view` object. The existing P/R has been updated to reflect that suggestion.

[2017-07 Toronto Wed Issue Prioritization]

Priority ; Marshall to investigate and if OK, will propose Tentatively Ready on reflector.

[2018-03-03: STL reported a related issue, LWG [3075](#).]

[2018-14: Wednesday night issues processing: both this and [3075](#) to status "Immediate".]

Proposed resolution:

This wording is relative to [N4656](#).

1. Edit 24.3.2 [\[basic.string\]](#), class template `basic_string` synopsis, as indicated:

[...]

// 21.3.2.2, construct/copy/destroy

[...]

```
template<class T>
explicit basic_string(basic_string_view<charT, traits> svconst T& t,
                    const Allocator& a = Allocator());
```

[...]

```
template<class T>
basic_string& operator=(basic_string_view<charT, traits> svconst T& t);
basic_string& operator=(const charT* s);
[...]
```

// 21.3.2.6, modifiers

[...]

```
template<class T>
basic_string& operator+=(basic_string_view<charT, traits> svconst T& t);
```

[...]

```
template<class T>
basic_string& append(basic_string_view<charT, traits> svconst T& t);
```

[...]

```
template<class T>
basic_string& assign(basic_string_view<charT, traits> svconst T& t);
```

```

[...]  

template<class T>  

basic_string& insert(size_type pos, basic_string_view<charT, traits> svconst T& t);  

[...]  

template<class T>  

basic_string& replace(size_type pos1, size_type n1,  

basic_string_view<charT, traits> svconst T& t);  

[...]  

template<class T>  

basic_string& replace(const_iterator i1, const_iterator i2,  

basic_string_view<charT, traits> svconst T& t);  

[...]  

// 21.3.2.7, string operations  

[...]  

template<class T>  

size_type find(basic_string_view<charT, traits> svconst T& t,  

size_type pos = 0) const noexcept;  

[...]  

template<class T>  

size_type rfind(basic_string_view<charT, traits> svconst T& t,  

size_type pos = npos) const noexcept;  

[...]  

template<class T>  

size_type find_first_of(basic_string_view<charT, traits> svconst T& t,  

size_type pos = 0) const noexcept;  

[...]  

template<class T>  

size_type find_last_of(basic_string_view<charT, traits> svconst T& t,  

size_type pos = npos) const noexcept;  

[...]  

template<class T>  

size_type find_first_not_of(basic_string_view<charT, traits> svconst T& t,  

size_type pos = 0) const noexcept;  

[...]  

template<class T>  

size_type find_last_not_of(basic_string_view<charT, traits> svconst T& t,  

size_type pos = npos) const noexcept;  

[...]  

template<class T>  

int compare(basic_string_view<charT, traits> svconst T& t) const noexcept;  

[...]  

template<class T>  

int compare(size_type pos1, size_type n1, basic_string_view<charT, traits> svconst T& t) const;  

[...]  


```

2. Edit 24.3.2.2 [string.cons] as indicated:

```

template<class T>  

explicit basic_string(basic_string_view<charT, traits> svconst T& t,  

const Allocator& a = Allocator());  


```

-9- Effects: ~~Same as basic_string(sv.data(), sv.size(), a).~~Creates a variable, sv, as if by
~~basic_string_view<charT, traits> sv = t;~~ and then behaves the same as ~~basic_string(sv.data(), sv.size(),~~
~~a).~~

-?- Remarks: This constructor shall not participate in overload resolution unless ~~is convertible v<const T&,~~
~~basic_string_view<charT, traits>> is true and is convertible v<const T&, const charT*> is false.~~

[...]

```

template<class T>  

basic_string& operator=(basic_string_view<charT, traits> svconst T& t);  


```

-25- Effects: Equivalent to:

```

{  

    basic_string_view<charT, traits> sv = t;  

    return assign(sv);  

}
```

-?- Remarks: This function shall not participate in overload resolution unless ~~is convertible v<const T&,~~
~~basic_string_view<charT, traits>> is true and is convertible v<const T&, const charT*> is false.~~

3. Edit 24.3.2.6.1 [string.op+=] as indicated:

```

template<class T>  

basic_string& operator+=(basic_string_view<charT, traits> svconst T& t);  


```

-3- Effects: ~~Creates a variable, sv, as if by basic_string_view<charT, traits> sv = t; and then c~~alls append(sv).

-4- Returns: *this.

-?- Remarks: This function shall not participate in overload resolution unless `is_convertible_v<const T&, basic_string_view<charT, traits>>` is true and `is_convertible_v<const T&, const charT*>` is false.

4. Edit 24.3.2.6.2 [\[string.append\]](#) as indicated:

```
template<class T>
basic_string& append(basic_string_view<charT, traits> svconst T& t);
```

-6- Effects: Equivalent to:

```
{
    basic_string_view<charT, traits> sv = t;
    return append(sv.data(), sv.size());
}
```

-?- Remarks: This function shall not participate in overload resolution unless `is_convertible_v<const T&, basic_string_view<charT, traits>>` is true and `is_convertible_v<const T&, const charT*>` is false.

5. Edit 24.3.2.6.3 [\[string.assign\]](#) as indicated:

```
template<class T>
basic_string& assign(basic_string_view<charT, traits> svconst T& t);
```

-8- Effects: Equivalent to:

```
{
    basic_string_view<charT, traits> sv = t;
    return assign(sv.data(), sv.size());
}
```

-?- Remarks: This function shall not participate in overload resolution unless `is_convertible_v<const T&, basic_string_view<charT, traits>>` is true and `is_convertible_v<const T&, const charT*>` is false.

6. Edit 24.3.2.6.4 [\[string.insert\]](#) as indicated:

```
template<class T>
basic_string& insert(size_type pos, basic_string_view<charT, traits> svconst T& t);
```

-5- Effects: Equivalent to:

```
{
    basic_string_view<charT, traits> sv = t;
    return insert(pos, sv.data(), sv.size());
}
```

-?- Remarks: This function shall not participate in overload resolution unless `is_convertible_v<const T&, basic_string_view<charT, traits>>` is true and `is_convertible_v<const T&, const charT*>` is false.

7. Edit 24.3.2.6.6 [\[string.replace\]](#) as indicated:

```
template<class T>
basic_string& replace(size_type pos1, size_type n1,
basic_string_view<charT, traits> svconst T& t);
```

-5- Effects: Equivalent to:

```
{
    basic_string_view<charT, traits> sv = t;
    return replace(pos1, n1, sv.data(), sv.size());
}
```

-?- Remarks: This function shall not participate in overload resolution unless `is_convertible_v<const T&, basic_string_view<charT, traits>>` is true and `is_convertible_v<const T&, const charT*>` is false.

[...]

```
template<class T>
basic_string& replace(const_iterator i1, const_iterator i2,
basic_string_view<charT, traits> svconst T& t);
```

-21- Requires: `[begin(), i1)` and `[i1, i2)` are valid ranges.

-22- Effects: Creates a variable, `sv`, as if by `basic_string_view<charT, traits> sv = t;` and then calls `replace(i1 - begin(), i2 - i1, sv)`.

-23- Returns: `*this`.

-?- Remarks: This function shall not participate in overload resolution unless `is_convertible_v<const T&, basic_string_view<charT, traits>>` is true and `is_convertible_v<const T&, const charT*>` is false.

8. Edit 24.3.2.7.2 [\[string.find\]](#) as indicated:

```
template<class T>  
size_type find(basic_string_view<charT, traits> svconst T& t, size_type pos = 0) const noexcept;
```

-1- *Effects:* Creates a variable, sv, as if by `basic_string_view<charT, traits> sv = t`; and then d~~D~~determines the lowest position xpos, if possible, such that both of the following conditions hold: [...]

-2- *Returns:* xpos if the function can determine such a value for xpos. Otherwise, returns npos.

-?- *Remarks:* This function shall not participate in overload resolution unless `is_convertible_v<const T&, basic_string_view<charT, traits>>` is true and `is_convertible_v<const T&, const charT*>` is false.

-?- *Throws:* Nothing unless the initialization of sv throws an exception.

9. Edit 24.3.2.7.3 [\[string.rfind\]](#) as indicated:

```
template<class T>  
size_type rfind(basic_string_view<charT, traits> svconst T& t, size_type pos = npos) const noexcept;
```

-1- *Effects:* Creates a variable, sv, as if by `basic_string_view<charT, traits> sv = t`; and then d~~D~~determines the highest position xpos, if possible, such that both of the following conditions hold: [...]

-2- *Returns:* xpos if the function can determine such a value for xpos. Otherwise, returns npos.

-?- *Remarks:* This function shall not participate in overload resolution unless `is_convertible_v<const T&, basic_string_view<charT, traits>>` is true and `is_convertible_v<const T&, const charT*>` is false.

-?- *Throws:* Nothing unless the initialization of sv throws an exception.

10. Edit 24.3.2.7.4 [\[string.find.first.of\]](#) as indicated:

```
template<class T>  
size_type find_first_of(basic_string_view<charT, traits> svconst T& t, size_type pos = 0) const noexcept;
```

-1- *Effects:* Creates a variable, sv, as if by `basic_string_view<charT, traits> sv = t`; and then d~~D~~determines the lowest position xpos, if possible, such that both of the following conditions hold: [...]

-2- *Returns:* xpos if the function can determine such a value for xpos. Otherwise, returns npos.

-?- *Remarks:* This function shall not participate in overload resolution unless `is_convertible_v<const T&, basic_string_view<charT, traits>>` is true and `is_convertible_v<const T&, const charT*>` is false.

-?- *Throws:* Nothing unless the initialization of sv throws an exception.

11. Edit 24.3.2.7.5 [\[string.find.last.of\]](#) as indicated:

```
template<class T>  
size_type find_last_of(basic_string_view<charT, traits> svconst T& t, size_type pos = npos) const noexcept;
```

-1- *Effects:* Creates a variable, sv, as if by `basic_string_view<charT, traits> sv = t`; and then d~~D~~determines the highest position xpos, if possible, such that both of the following conditions hold: [...]

-2- *Returns:* xpos if the function can determine such a value for xpos. Otherwise, returns npos.

-?- *Remarks:* This function shall not participate in overload resolution unless `is_convertible_v<const T&, basic_string_view<charT, traits>>` is true and `is_convertible_v<const T&, const charT*>` is false.

-?- *Throws:* Nothing unless the initialization of sv throws an exception.

12. Edit 24.3.2.7.6 [\[string.find.first.not.of\]](#) as indicated:

```
template<class T>  
size_type find_first_not_of(basic_string_view<charT, traits> svconst T& t,  
                           size_type pos = 0) const noexcept;
```

-1- *Effects:* Creates a variable, sv, as if by `basic_string_view<charT, traits> sv = t`; and then d~~D~~determines the lowest position xpos, if possible, such that both of the following conditions hold: [...]

-2- *Returns:* xpos if the function can determine such a value for xpos. Otherwise, returns npos.

-?- *Remarks:* This function shall not participate in overload resolution unless `is_convertible_v<const T&, basic_string_view<charT, traits>>` is true and `is_convertible_v<const T&, const charT*>` is false.

-?- *Throws:* Nothing unless the initialization of sv throws an exception.

13. Edit 24.3.2.7.7 [\[string.find.last.not.of\]](#) as indicated:

```
template<class T>
size_type find_last_not_of(basic_string_view<charT, traits> svconst T& t,
                          size_type pos = npos) const noexcept;
```

-1- *Effects*: Creates a variable, `sv`, as if by `basic_string_view<charT, traits> sv = t;` and then determines the highest position `xpos`, if possible, such that both of the following conditions hold: [...]

-2- *Returns*: `xpos` if the function can determine such a value for `xpos`. Otherwise, returns `npos`.

Remarks: This function shall not participate in overload resolution unless `is_convertible_v<const T&, basic_string_view<charT, traits>>` is true and `is_convertible_v<const T&, const charT*>` is false.

Throws: Nothing unless the initialization of `sv` throws an exception.

14. Edit 24.3.2.7.9 [\[string.compare\]](#) as indicated:

```
template<class T>
int compare(basic_string_view<charT, traits> svconst T& t) const noexcept;
```

-1- *Effects*: Creates a variable, `sv`, as if by `basic_string_view<charT, traits> sv = t;` and then determines the effective length `rlen` of the strings to compare as the smaller of `size()` and `sv.size()`. The function then compares the two strings by calling `traits::compare(data(), sv.data(), rlen)`.

-2- *Returns*: The nonzero result if the result of the comparison is nonzero. Otherwise, returns a value as indicated in Table 63.

[...]

Remarks: This function shall not participate in overload resolution unless `is_convertible_v<const T&, basic_string_view<charT, traits>>` is true and `is_convertible_v<const T&, const charT*>` is false.

Throws: Nothing unless the initialization of `sv` throws an exception.

```
template<class T>
int compare(size_type pos1, size_type n1, basic_string_view<charT, traits> svconst T& t) const;
```

-3- *Effects*: Equivalent to:

```
{
    basic_string_view<charT, traits> sv = t;
    return basic_string_view<charT, traits>(data(), size()).substr(pos1, n1).compare(sv);
}
```

Remarks: This function shall not participate in overload resolution unless `is_convertible_v<const T&, basic_string_view<charT, traits>>` is true and `is_convertible_v<const T&, const charT*>` is false.

3075⁽ⁱ⁾. `basic_string` needs deduction guides from `basic_string_view`

Section: 24.3.2 [\[basic.string\]](#), 24.3.2.2 [\[string.cons\]](#) **Status:** [Immediate](#) **Submitter:** Stephan T. Lavavej **Opened:** 2018-03-03 **Last modified:** 2018-03-16

Priority: Not Prioritized

View other [active issues](#) in [\[basic.string\]](#).

View all other [issues](#) in [\[basic.string\]](#).

View all issues with [Immediate](#) status.

Discussion:

The Proposed Resolution for LWG [2946](#) appears to be correct and we've implemented it in MSVC, but it worsens a pre-existing problem with `basic_string` class template argument deduction.

The following `s1` and `s2` compiled in C++17 before LWG 2946's PR, fail to compile after LWG 2946's PR, and are fixed by my PR:

```
basic_string s1("cat"sv);
basic_string s2("cat"sv, alloc);
```

The following `s4` failed to compile in C++17, and is fixed by my PR:

```
// basic_string s3("cat"sv, 1, 1);
basic_string s4("cat"sv, 1, 1, alloc);
```

(`s3` failed to compile in C++17, and would be fixed by my PR, but it is affected by a pre-existing and unrelated ambiguity which I am not attempting to fix here.)

As C++17 and LWG 2946's PR introduced templated constructors for `basic_string` from `basic_string_view`, we need to add corresponding deduction guides.

The constructors take `const T&` that's convertible to `basic_string_view` (the additional constraint about not converting to `const charT*` is irrelevant here). However, CTAD can't deduce `charT` and traits from arbitrary user-defined types, so the deduction guides need `T` to be exactly `basic_string_view`.

Additionally, we need to handle the `size_type` parameters in the same way that the unordered containers do. This PR has been implemented in MSVC.

[2018-14: Wednesday night issues processing: both this and [2946](#) to status "Immediate".]

Proposed resolution:

This wording is relative to [N4727](#).

1. Edit 24.3.2 [\[basic.string\]](#), class template `basic_string` synopsis, as indicated:

```
[...]
template<class InputIterator,
        class Allocator = allocator<typename iterator_traits<InputIterator>::value_type>>
    basic_string(InputIterator, InputIterator, Allocator = Allocator())
    -> basic_string<typename iterator_traits<InputIterator>::value_type,
        char_traits<typename iterator_traits<InputIterator>::value_type>,
        Allocator>;

template<class charT,
        class traits,
        class Allocator = allocator<charT>>
    explicit basic_string(basic_string_view<charT, traits>, const Allocator& = Allocator()),
    -> basic_string<charT, traits, Allocator>;

template<class charT,
        class traits,
        class Allocator = allocator<charT>>
    basic_string(basic_string_view<charT, traits>, typename see below::size_type, typename see below::size_type,
        const Allocator& = Allocator()),
    -> basic_string<charT, traits, Allocator>;
}
```

-?- A `size_type` parameter type in a `basic_string` deduction guide refers to the `size_type` member type of the type deduced by the deduction guide.

2. Edit 24.3.2.2 [\[string.cons\]](#) as indicated:

```
template<class InputIterator,
        class Allocator = allocator<typename iterator_traits<InputIterator>::value_type>>
    basic_string(InputIterator, InputIterator, Allocator = Allocator())
    -> basic_string<typename iterator_traits<InputIterator>::value_type,
        char_traits<typename iterator_traits<InputIterator>::value_type>,
        Allocator>;
```

-25- Remarks: Shall not participate in overload resolution if `InputIterator` is a type that does not qualify as an input iterator, or if `Allocator` is a type that does not qualify as an allocator (26.2.1 [\[container.requirements.general\]](#)).

```
template<class charT,
        class traits,
        class Allocator = allocator<charT>>
    explicit basic_string(basic_string_view<charT, traits>, const Allocator& = Allocator()),
    -> basic_string<charT, traits, Allocator>;

template<class charT,
        class traits,
        class Allocator = allocator<charT>>
    basic_string(basic_string_view<charT, traits>, typename see below::size_type, typename see below::size_type,
        const Allocator& = Allocator()),
    -> basic_string<charT, traits, Allocator>;
```

-?- Remarks: Shall not participate in overload resolution if `Allocator` is a type that does not qualify as an allocator (26.2.1 [\[container.requirements.general\]](#)).